

# TIM2 기반 HAL Time Base 타이머 예제 분석

STM32G0B1RE · HAL\_TimeBase · Prescaler · 1 ms Tick Interrupt



클럭 분주 계산 · Tick 생성 · HAL\_Delay() · Suspend/Resume Tick

# 학습 목표

TIM2를 HAL 시간 기준으로 사용하는 구조와 계산식을 이해합니다.

STM32 HAL time base가 무엇이고, 왜 1 ms tick이 필요한지 설명할 수 있다.

SYSCLK → PCLK1 → TIM2CLK → Counter clock → Update event까지의 분주 구조를 설명할 수 있다.

Prescaler(PSC)와 Period/Auto-Reload(ARR)의 의미와 계산 방법을 이해한다.

TIM2 update interrupt가 발생하면 HAL\_IncTick()으로 uwTick이 증가하는 흐름을 설명한다.

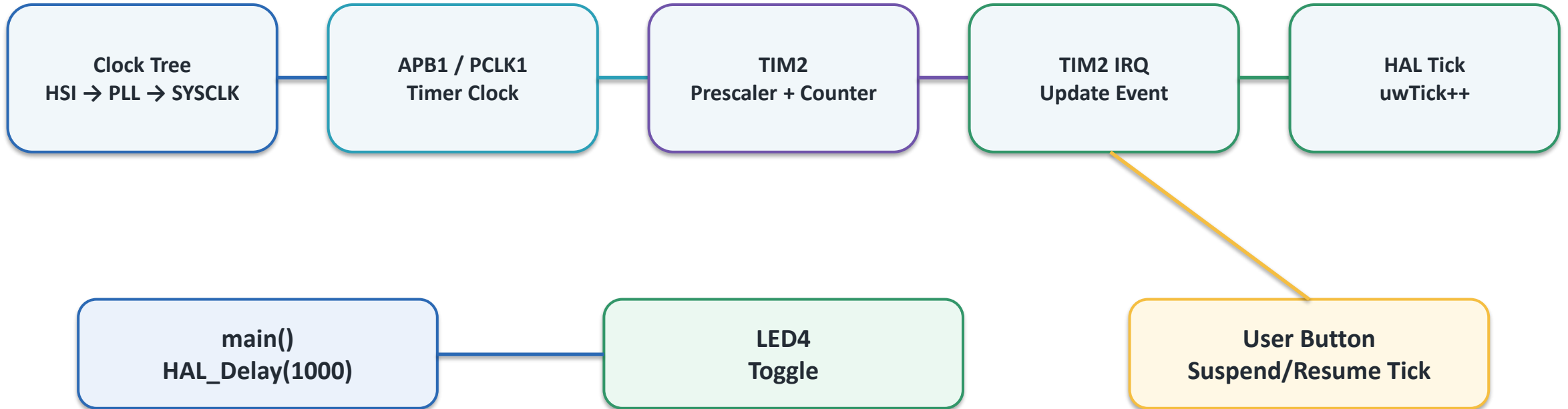
HAL\_Delay(1000)이 실제 1초 지연으로 동작하는 원리를 설명한다.

User Button EXTI로 HAL\_SuspendTick() / HAL\_ResumeTick()을 전환하는 동작을 분석한다.

핵심 질문: “1 ms는 어디서 만들어지고, HAL\_Delay()는 무엇을 기준으로 시간을 재는가?”

# 예제의 전반적인 Architecture

TIM2가 HAL의 시간 기준을 만들고, main loop는 그 tick을 이용해 LED를 1초마다 토글합니다.



구조적으로 이 예제는 “타이머가 1 ms 기준 시간을 만들고, 애플리케이션은 HAL\_Delay()와 버튼 인터럽트로 그 기준 시간의 흐름을 제어”하는 예제입니다.

# main() 함수 실행 흐름

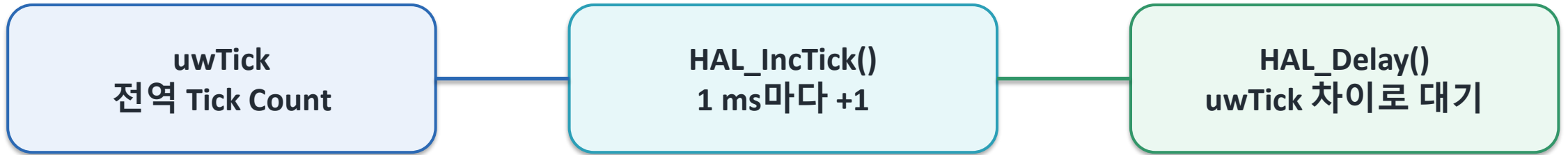
초기화 이후 while(1)에서 HAL\_Delay(1000)과 LED Toggle을 반복합니다.



HAL\_Delay(1000)은 TIM2 interrupt로 증가하는 uwTick 값을 기준으로 1000 ms를 기다립니다.

# HAL Time Base란?

HAL\_Delay(), timeout 처리, 주기 측정의 기준이 되는 1 ms tick입니다.



기본 HAL은 SysTick을 time base로 사용하는 경우가 많지만, 이 예제는 TIM2를 HAL time base로 사용한다.

TIM2 update interrupt가 1 ms마다 발생하고, callback에서 HAL\_IncTick()을 호출한다.

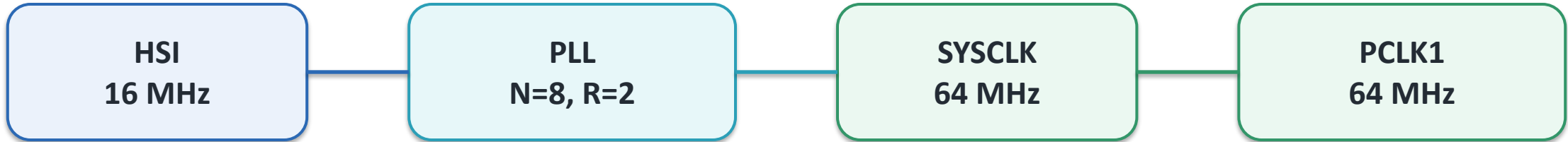
따라서 HAL\_Delay(1000)은 TIM2 tick이 1000번 증가할 때까지 기다리는 구조이다.

HAL\_SuspendTick()으로 TIM2 update interrupt를 끄면 uwTick 증가가 멈추고, HAL\_Delay()의 시간 진행도 멈춘다.

**강의 포인트:** HAL\_Delay()는 CPU 내부의 '마법의 타이머'가 아니라, HAL tick 변수의 증가를 기다리는 함수입니다.

# 클럭 설정 분석

HSI 16 MHz를 PLL로 64 MHz SYSCLK로 만들고, APB1 분주 없이 TIM2에 공급합니다.



## PLL 계산

$$\text{VCO} = \text{HSI} / \text{PLLM} \times \text{PLLN} = 16 \text{ MHz} / 1 \times 8 = 128 \text{ MHz}$$

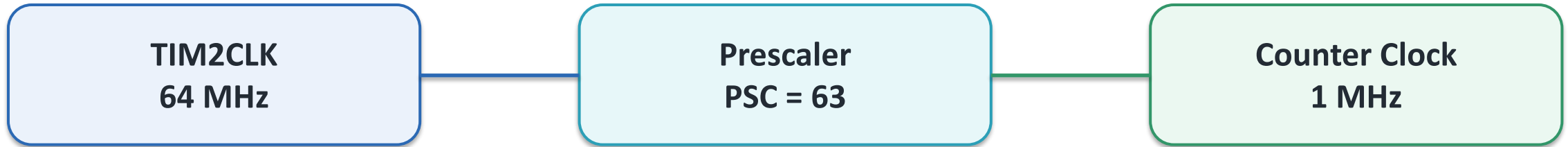
$$\text{SYSCLK} = \text{VCO} / \text{PLLR} = 128 \text{ MHz} / 2 = 64 \text{ MHz}$$

APB1CLKDivider = RCC\_HCLK\_DIV1 이므로 PCLK1 = HCLK = 64 MHz

프리스케일러(Prescaler, 전치분주기) : 입력된 클럭 주파수를 정수 비율로 나누어 더 느린 주파수로 낮춰주는 전자 회로 장치

## Prescaler 계산

TIM2 counter clock을 1 MHz로 만들기 위해 Prescaler 값을 계산합니다.



### PSC 계산식

$$\text{Counter Clock} = \text{TIM2CLK} / (\text{PSC} + 1)$$

$$\text{PSC} = (\text{TIM2CLK} / \text{원하는 Counter Clock}) - 1 = (64,000,000 / 1,000,000) - 1 = 63$$

$$\text{Counter 1 tick 시간} = 1 / 1 \text{ MHz} = 1 \mu\text{s}$$

## 프리스케일러 동작

타이머가 실제로 클럭을 나누는 실제 분주비는 설정값 + 1이 됩니다. 레지스터에 0을 넣으면 1분주(나누지 않음)가 되기 때문입니다.

$$\text{타이머 카운터 클럭 (Hz)} = \frac{\text{타이머 공급클럭 (Hz)}}{PSC\text{설정값} + 1}$$

### 3. 계산 예시

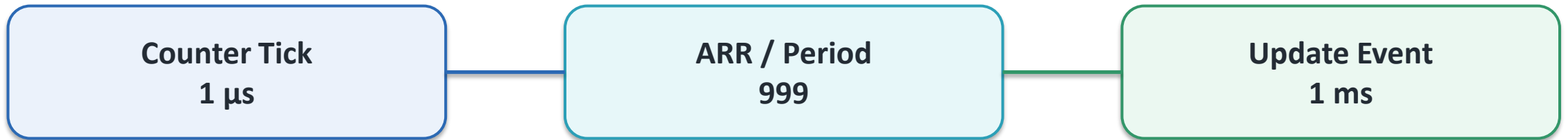
만약 STM32G0의 시스템 클럭  $f_{clk}$ 이 **64MHz (64,000,000 Hz)**로 동작하고 있다고 가정할 때,

- **예시 A (1000분주를 하고 싶을 때):**
  - PSC 레지스터에 999를 입력 -> 실제 분주비 =  $999 + 1 = 1000$  마다 분주
  - 타이머 카운터 클럭 =  $64,000,000 \div 1000 = 64,000 \text{ Hz}$  (1초에 64,000번 카운트 증가)
- **예시 B (정확히 1us(마이크로초)마다 카운트를 1씩 증가하고자 할 경우):**
  - 1us 마다 카운트가 1증가하려면 타이머 클럭이 **1MHz(1,000,000 Hz)**가 되어야 함.
  - $(64,000,000 \div \text{실제 분주비}) = 1,000,000 \Rightarrow \text{실제 분주비} = 64$
  - PSC 레지스터에 63을 설정하면 됨 ( $63 + 1 = 64$ ).



## Period / Auto-Reload 계산

1 MHz counter가 1000번 세면 1 ms update event가 발생합니다.



### Period 계산식

$$\begin{aligned}\text{Update 주기} &= (\text{PSC} + 1) \times (\text{ARR} + 1) / \text{TIM2CLK} \\ &= 64 \times 1000 / 64,000,000 = 0.001 \text{ s} = 1 \text{ ms}\end{aligned}$$

HAL 코드: `htim2.Init.Period = (1000000 / 1000) - 1 = 999`

# HAL\_InitTick() 코드 분석

TIM2를 1 ms tick interrupt source로 설정하는 핵심 함수입니다.

## NVIC 설정

```
HAL_NVIC_SetPriority(TIM2_IRQn, TickPriority, 0)  
HAL_NVIC_EnableIRQ(TIM2_IRQn)
```

## TIM2 clock enable

```
__HAL_RCC_TIM2_CLK_ENABLE()
```

## Timer clock 계산

APB1 prescaler가 DIV10이면  $TIM2CLK = PCLK1$   
그 외에는  $TIM2CLK = 2 \times PCLK1$

## PSC / ARR 설정

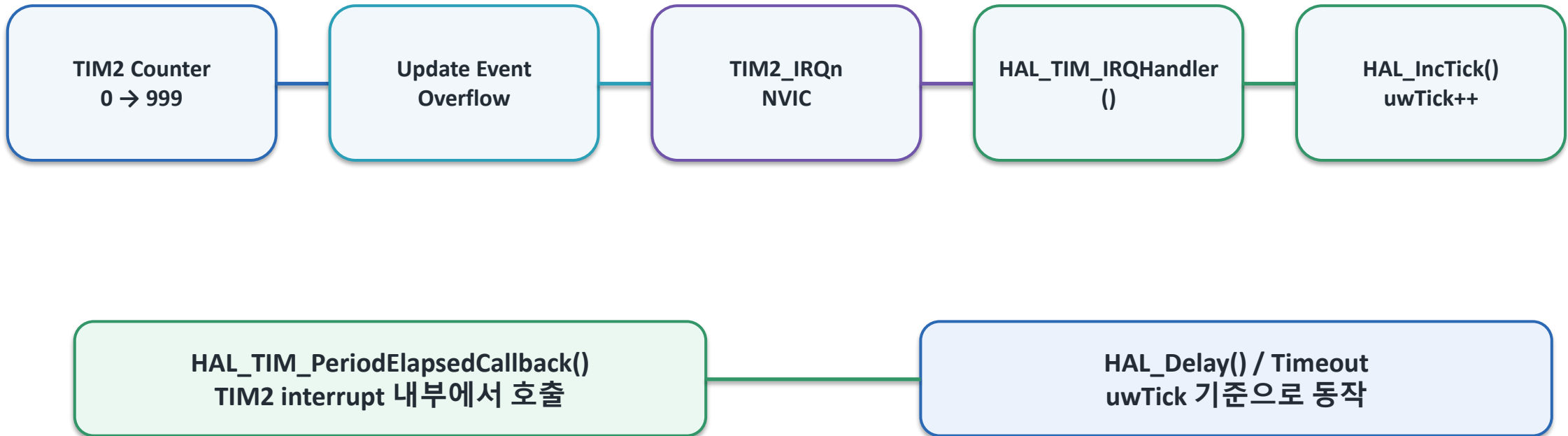
**Prescaler = 63, Period = 999**

## 타이머 시작

```
HAL_TIM_Base_Init(&htim2)  
HAL_TIM_Base_Start_IT(&htim2)
```

# TIM2 Interrupt 흐름

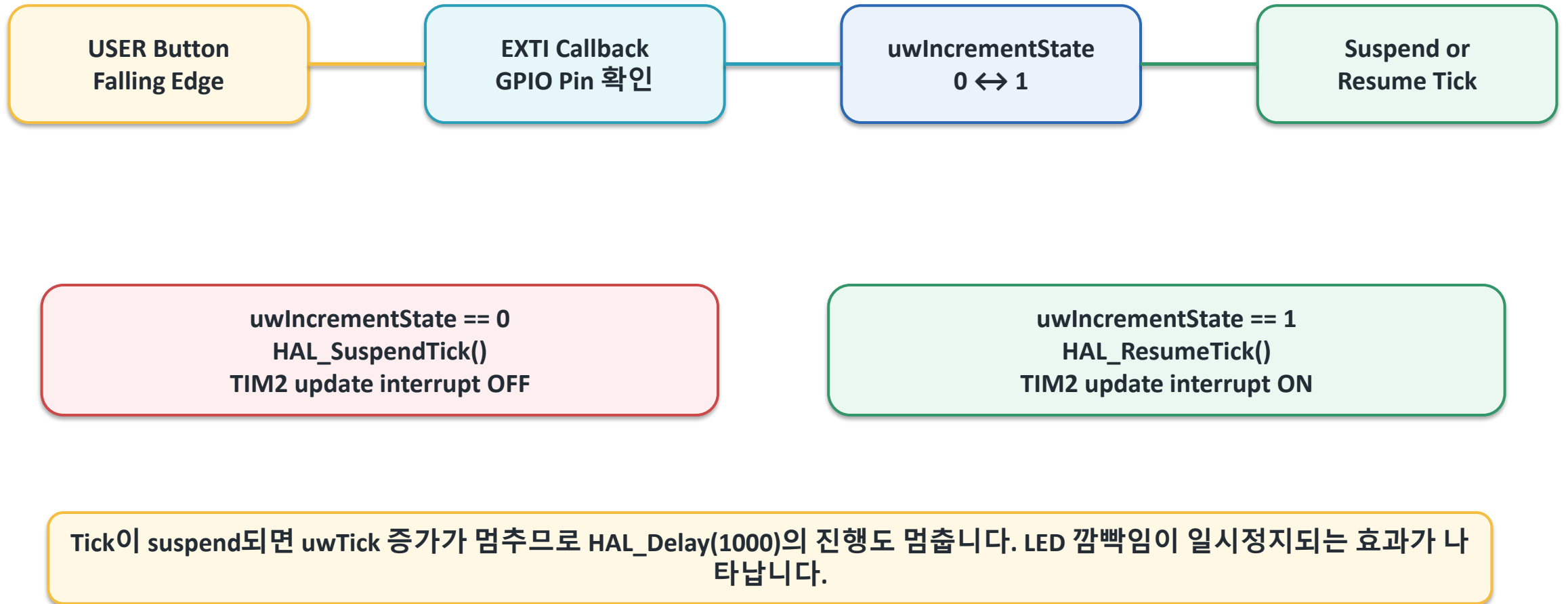
Update interrupt가 발생하면 HAL callback을 통해 uwTick을 증가시킵니다.



**강의 포인트:** Timer interrupt 자체가 LED를 토글하는 것이 아니라, HAL tick을 증가시키고 main loop의 HAL\_Delay()가 그 tick을 사용합니다.

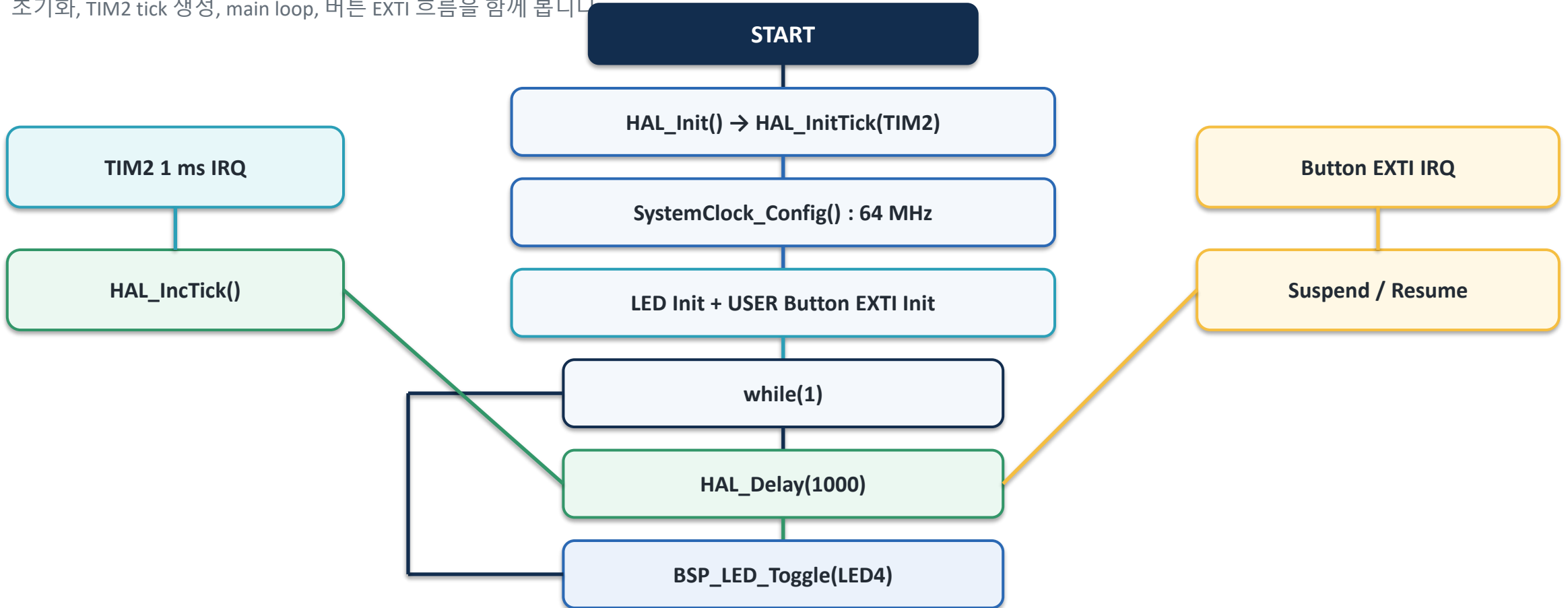
# User Button으로 Tick Suspend / Resume

버튼이 눌릴 때마다 TIM2 update interrupt를 끄거나 다시 켭니다.



# 전체 프로그램 Flow Chart

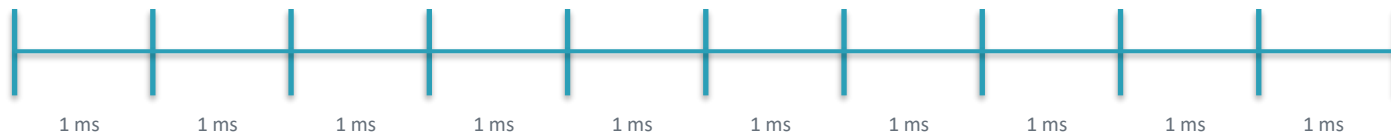
초기화, TIM2 tick 생성, main loop, 버튼 EXTI 흐름을 함께 봅니다.



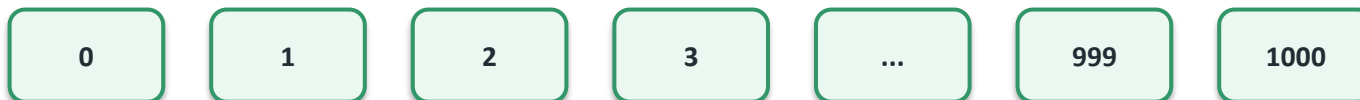
# Timing Diagram으로 이해하기

TIM2 update는 1 ms마다 발생하고, HAL\_Delay(1000)은 tick 1000개를 기다립니다.

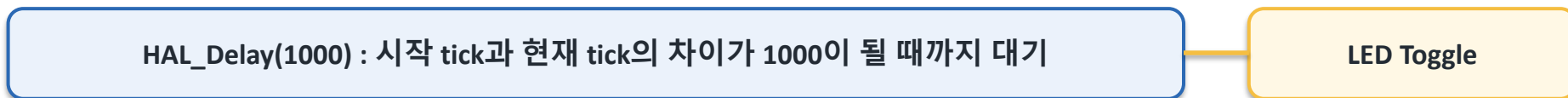
TIM2 Update



uwTick



HAL\_Delay



단위 사이클 타임: TIM2 counter tick은 1  $\mu$ s, update event는 1000 counter tick마다 1 ms

# 타이머 계산 공식 정리

학생들이 직접 다른 주기의 타이머를 설계할 때 사용하는 핵심 공식입니다.

$$\text{Timer Counter Frequency} = \text{TIMxCLK} / (\text{PSC} + 1)$$

$$\text{Counter Tick Time} = 1 / \text{Timer Counter Frequency}$$

$$\text{Update Period} = (\text{PSC} + 1) \times (\text{ARR} + 1) / \text{TIMxCLK}$$

$$\text{ARR} = \text{원하는 Update Period} \times \text{Counter Frequency} - 1$$

이 예제: TIMxCLK=64 MHz, PSC=63, Counter=1 MHz, ARR=999, Update Period=1 ms

# 주요 함수 기능 정리

코드 설명 시 함수의 책임을 분리해서 설명하면 전체 구조가 명확해집니다.

HAL\_Init()

HAL 기본 초기화, time base 초기화를 위해 HAL\_InitTick() 호출

SystemClock\_Config()

HSI + PLL로 SYSCLK 64 MHz 설정, FLASH latency 설정

HAL\_InitTick()

TIM2를 1 ms HAL time base interrupt로 구성

HAL\_TIM\_Base\_Init()

TIM2 기본 타이머 설정값을 레지스터에 반영

HAL\_TIM\_Base\_Start\_IT()

TIM2 update interrupt 방식으로 타이머 시작

HAL\_TIM\_PeriodElapsedCallback()

TIM2 update interrupt 이후 호출, HAL\_IncTick() 수행

HAL\_SuspendTick()

TIM2 update interrupt 비활성화 → uwTick 증가 중지

HAL\_ResumeTick()

TIM2 update interrupt 활성화 → uwTick 증가 재개

HAL\_GPIO\_EXTI\_Falling\_Callback()

USER 버튼 falling edge에서 suspend/resume 상태 전환

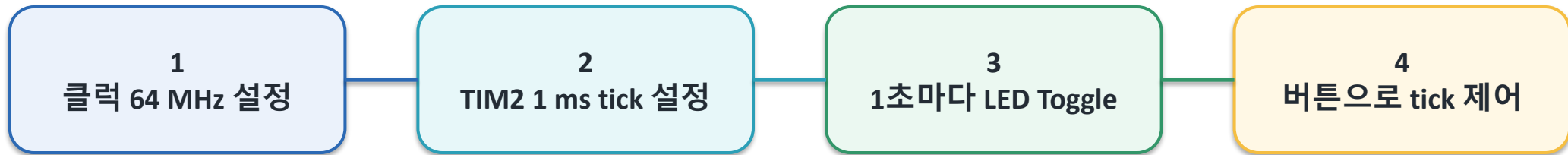
HAL\_Delay(1000)

uwTick 기준으로 1000 ms 지연



# 알고리즘 요약

이 예제를 한 문장으로 설명하면 “TIM2로 HAL tick을 만들고, 버튼으로 tick 흐름을 제어하는 프로그램”입니다.



Timer는 내부 counter가 clock을 세다가 ARR에 도달하면 update event를 발생시킨다.

Update event interrupt는 HAL\_TIM\_PeriodElapsedCallback()으로 연결되고, 여기서 HAL\_IncTick()이 실행된다.

HAL\_Delay()는 uwTick이 증가하는 것을 기다리므로, time base interrupt가 멈추면 delay 진행도 멈춘다.

USER 버튼 EXTI는 HAL\_SuspendTick()과 HAL\_ResumeTick()을 번갈아 호출해 LED 깜빡임을 정지/재개한다.

수업 실습 확장: ARR을 499로 바꾸면 0.5 ms tick, 1999로 바꾸면 2 ms tick이 됩니다.

# 실습 및 디버깅 포인트

학생들이 직접 레지스터와 변수 변화를 관찰하도록 유도합니다.

Watch 창에서 uwTick 값이 1 ms마다 증가하는지 확인한다.

TIM2 PSC=63, ARR=999 설정이 적용되었는지 디버거로 확인한다.

HAL\_TIM\_PeriodElapsedCallback()에 breakpoint를 걸어 주기적 interrupt를 확인한다.

USER 버튼을 누른 후 uwIncrementState가 0↔1로 바뀌는지 확인한다.

Tick suspend 상태에서 LED toggle이 멈추는지 관찰한다.

PSC 또는 ARR 값을 바꾸고 LED toggle 주기가 어떻게 달라지는지 실험한다.

**정상 동작**  
**LED4가 약 1초마다 Toggle**

**버튼 1회 누름**  
**TIM2 Update Interrupt OFF**

**버튼 다시 누름**  
**TIM2 Update Interrupt ON**

**주의: ISR/Callback 안에서는 긴 지연 작업을 피한다.**

## SUMMARY

# TIM2로 만드는 1 ms HAL Tick

Clock → Prescaler → Counter → Update Interrupt → HAL\_IncTick() → HAL\_Delay()

Clock 계산  
64 MHz TIM2CLK

PSC 계산  
64→1 MHz

ARR 계산  
1000 tick = 1 ms

Button EXTI  
Suspend/Resume

Timer 강의의 핵심은 숫자 계산입니다.  
클럭이 몇 Hz인지, 한 counter tick이 몇 초인지, update event가 언제 발생하는지 계산해야 합니다.